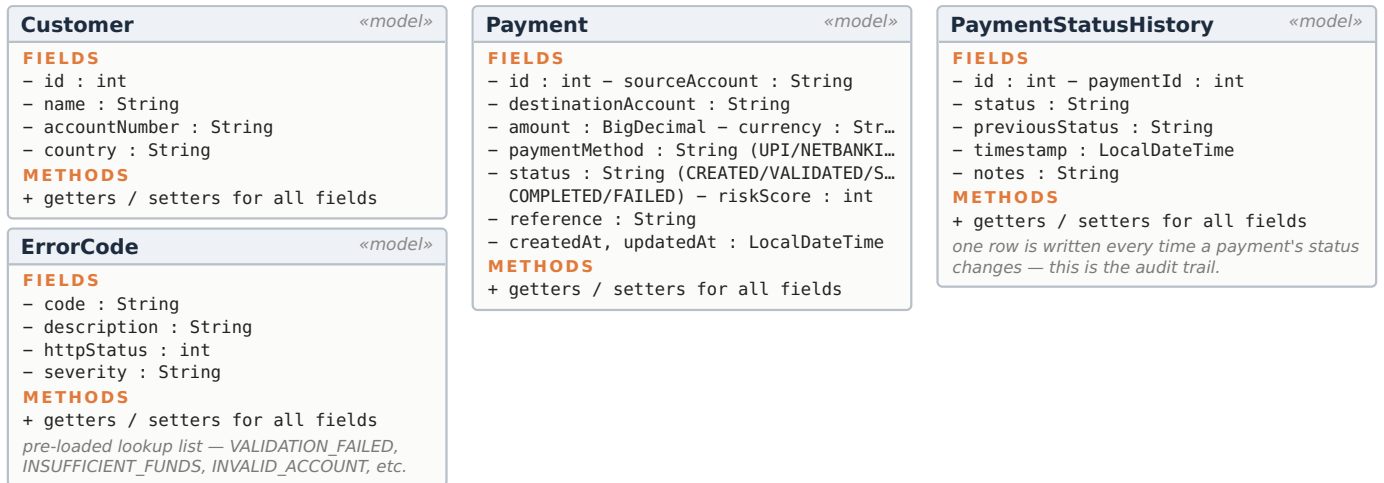


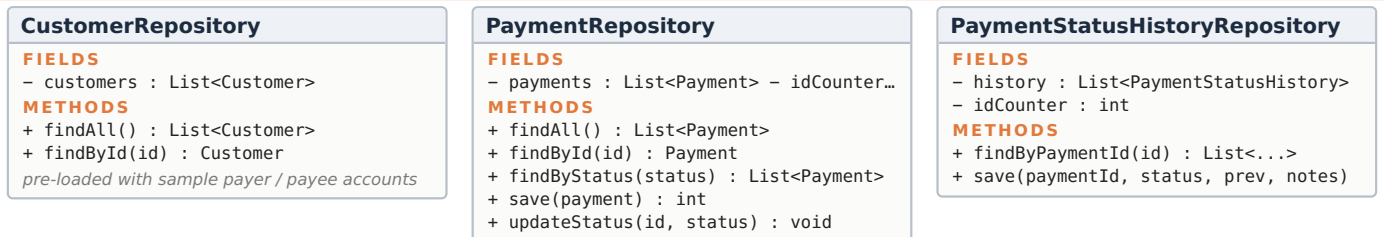
# Payments Processing System — Class Diagram

Every class in the app, shown as a block with its fields and methods — controller methods are shown as their actual HTTP endpoints. Build top to bottom: Models first, then Repositories, then Services, then Controllers, then wire it all up with the Application class.

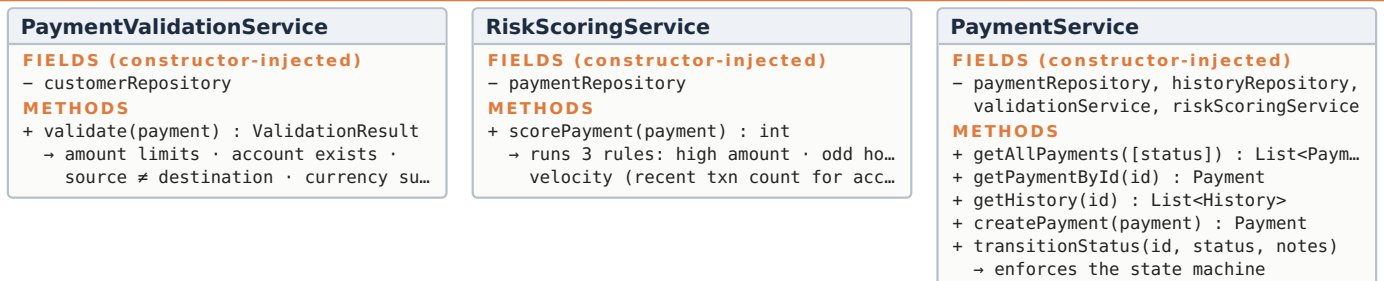
## 1 MODEL LAYER — plain data objects, build these first



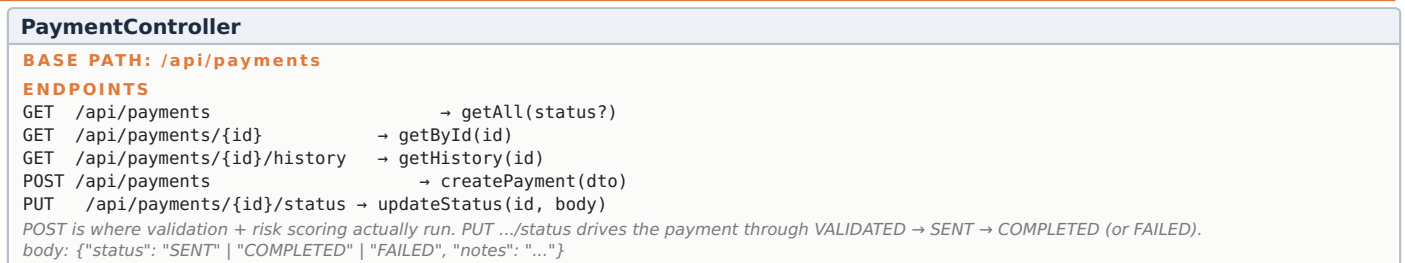
## 2 REPOSITORY LAYER — @Repository, owns persistence for one model each



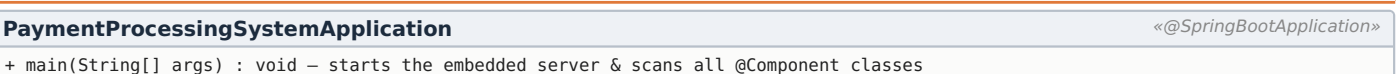
## 3 SERVICE LAYER — @Service, all business logic lives here



## 4 CONTROLLER LAYER — @RestController, methods = HTTP endpoints



## 5 ENTRY POINT — wires everything together and starts the server



Model · Repository · Service · Controller · Application entry point — field + public method «...» Spring stereotype annotation

# Payments Processing System — Layered Architecture

The layered architecture used throughout this project. Each layer has exactly one job — the browser never talks to the database directly, and business rules never live in the controller. Three cross-cutting concerns (bottom row) touch every layer without belonging to any single one of them.

## Frontend — index.html

PRESENTATION LAYER

HTML5 + CSS3 + Bootstrap 5 + Vanilla JavaScript · fetch() calls to /api/\*, refreshes list every 8s

HTTP · JSON (REST)

## @RESTCONTROLLER LAYER

CONTROLLER LAYER

**PaymentController** — GET/POST /api/payments · GET .../history · PUT .../status · @Valid on request DTOs

DTO → service method call

## @SERVICE LAYER — BUSINESS LOGIC

SERVICE LAYER

**PaymentService** (lifecycle orchestration) — creates payments, drives CREATED→VALIDATED→SENT→COMPLETED, writes history rows  
**PaymentValidationService** — amount / account / currency rules · **RiskScoringService** — high amount · odd hours · velocity → risk score

JdbcTemplate calls

## @REPOSITORY LAYER — JDBCTEMPLATE

REPOSITORY LAYER

**CustomerRepository** — findAll · findById · **PaymentRepository** — save (KeyHolder) · updateStatus · findByStatus · countRecent  
**PaymentStatusHistoryRepository** — save · findByPaymentId

SQL over JDBC connection

## MySQL — paymentsdb

DATA LAYER

Tables: customer · payment · payment\_status\_history (schema.sql / data.sql run on every startup)

CROSS-CUTTING CONCERNS — touch every layer above, owned by none of them

### DTOs + Bean Validation

@NotNull, @DecimalMin, @Pattern on request DTOs, checked via @Valid

### GlobalExceptionHandler

@RestControllerAdvice catches validation / not-found / invalid-transition / unexpected errors → clean JSON

### SLF4J / Logback Logging

Logger per class · DEBUG reads, INFO status transitions, WARN on a fired risk rule, ERROR on failures

| COMPONENT                        | LAYER         | RESPONSIBILITY                                                                                                   |
|----------------------------------|---------------|------------------------------------------------------------------------------------------------------------------|
| index.html                       | Frontend      | Renders the dashboard, submits payments, polls the API, never talks to MySQL directly.                           |
| PaymentController                | Controller    | Exposes REST endpoints, validates incoming DTOs with @Valid, delegates everything else to a service.             |
| PaymentService                   | Service       | Owns the payment lifecycle state machine and orchestrates validation, risk scoring, and history writes.          |
| PaymentValidationService         | Service       | Runs business validation rules — amount limits, account checks, currency support, source ≠ destination.          |
| RiskScoringService               | Service       | Owns the rule-based risk score — high amount, odd hours, velocity — feeding basic fraud-awareness into Sprint 1. |
| *Repository classes              | Repository    | Hand-written SQL via JdbcTemplate — no JPA/Hibernate magic, so freshers see every query.                         |
| MySQL (paymentsdb)               | Data          | Stores customer, payment, and payment_status_history tables.                                                     |
| DTOs, Exception Handler, Logging | Cross-cutting | Shape the API contract, turn errors into clean JSON, and make every status transition traceable in the console.  |

# Request Interaction Flow — Creating a Payment

What actually happens, in order, when a user clicks "Submit Payment" on the dashboard. Solid arrows are calls; dashed arrows are returns. This is the single best diagram to walk through live, one arrow at a time, in your session.

Browser  
DashboardPayment  
ControllerPaymentService /  
ValidationService /  
RiskScoringServiceRepository  
Layer

MySQL

- 1 POST /api/payments (PaymentRequestDTO)
- 2 @Valid runs Bean Validation on the request DTO
- 3 createPayment(requestDTO)
- 4 validate(payment) — amount limits · account exists · source ≠ destination · currency supported
- 5 save(payment) with status = CREATED → INSERT INTO payment
- 6 INSERT ... (PreparedStatement + KeyHolder)
- 7 generated payment id returned
- 8 save(paymentId, "CREATED", null, "Payment submitted") → first history row
- 9 scorePayment(payment) — countRecent(sourceAccount) for the velocity rule
- 10 SELECT COUNT(\*) FROM payment WHERE source\_account = ? AND created\_at > ?
- 11 recent payment count returned
- 12 Evaluate risk rules — high amount · odd hours · velocity  
*Each triggered rule → log.warn(...) + adds to risk score (capped at 100)*

alt [ validation passed ]

- 13 updateStatus(id, "VALIDATED") → UPDATE payment SET status, risk\_score
- 14 save(paymentId, "VALIDATED", "CREATED", "Validation checks passed") → INSERT payment\_status\_history

else [ validation failed ]

- 13b updateStatus(id, "FAILED") + save history row with errorCode & notes — payment never reaches SENT

- 15 PaymentResponseDTO (status: VALIDATED | FAILED, riskScore, errorCode?)

- 16 201 Created — JSON response

*Dashboard updates: toast + table refresh; status badge colored by state, history timeline appended*

**Solid arrow** = synchronous call    **Dashed arrow** = return value    **Dashed box** = conditional (alt) branch

Later, **PUT /api/payments/{id}/status** follows the same Controller → Service → Repository → MySQL path to move a payment from VALIDATED → SENT → COMPLETED (or to FAILED at any stage), writing one new payment\_status\_history row per transition and rejecting any transition that isn't in the state machine.